Solver-Free Decision-Focused Learning via Neural Optimization Layers for Edge Selection

Xinda Zheng[a,b,*], Frits de Nijs[a,b] and Edward Lam[a,b]

[a]Department of Data Science and AI, Faculty of Information Technology, Monash University, Clayton, 3800, Victoria, Australia
[b]ARC Training Centre in Optimisation Technologies, Integrated Methodologies, and Applications (OPTIMA), Melbourne, 3000, Victoria, Australia

ARTICLE INFO

Keywords:
Decision-focused learning
Predict then optimize
Optimization under uncertainty
Self-supervised learning
Learning to optimize
Combinatorial optimization

ABSTRACT

Decision-focused learning trains prediction models to improve downstream decisions, but many methods require costly solver calls or optimal-solution labels. We propose a solver-free decision-focused learning framework for NP-hard edge-selection problems with uncertain parameters in the objective, the constraints, or both. A neural optimization layer is first trained without solution labels, using problem objectives and constraint penalties. Graph-informed edge scoring and iterative refinement encourage concentrated and structurally consistent continuous decisions. The layer is then frozen, and a prediction model is trained through its continuous output using a problem-specific loss evaluated under observed parameters. At deployment, the predictor, optimization layer, and a lightweight decoder produce decisions without solving the original NP-hard problem. Experiments on the asymmetric traveling salesperson and endpoint-free orienteering problems show consistently better decision quality than prediction-focused learning and improvements over most evaluated decision-focused baselines. Our framework requires less training time than the evaluated decision-focused baselines, even when optimization-layer pretraining is included. Neural deployment also produces decisions faster than solver-based deployment, offering a better quality-efficiency trade-off. The layer also reduces training cost in existing decision-focused methods when used as an approximate optimization oracle.

arXiv:2611.09560v1 [cs.LG] 6 Nov 2026

1. Introduction

Many real-world optimization tasks involve making decisions before all relevant information is available. In the predict-then-optimize paradigm, unknown problem parameters are estimated from available context, and an optimization problem is then solved using the estimates [1]. For example, a route may be planned a day in advance using predicted travel times. These uncertain parameters may affect the objective, the constraints, or both: travel times can determine a route's total cost as well as whether it satisfies a travel budget. However, predictors are typically trained to minimize statistical losses such as mean squared error, which measure prediction errors without accounting for their impact on downstream decisions. A large error may leave the optimal decision unchanged, while a small error near a decision boundary can lead to a decision that performs poorly under the true parameters [2]. Decision-focused learning (DFL) addresses this mismatch by training predictors to improve downstream decision quality [3].

A common approach in DFL achieves this alignment by defining the training loss through solutions to the downstream optimization problem. Evaluating these losses and their gradients requires repeated solver calls [4, 5], which can be particularly costly for NP-hard edge-selection problems such as the traveling salesperson and orienteering problems. Other approaches avoid repeatedly solving the original problem during predictor updates but require optimal reference solutions [6, 7]. However, real-world datasets often contain observed parameters rather than corresponding optimal solutions, so these reference solutions must be computed through potentially costly optimization. At deployment, solver-based methods must still solve an optimization problem for each new prediction [8], which can hinder applications requiring frequent or time-sensitive decisions.

Reducing computational cost is only part of the challenge. Decision feedback must also account for uncertainty in the constraints. DFL formulations that assume a fixed feasible set do not directly address prediction errors that change decision feasibility [9, 5, 6]. In edge-selection problems with uncertain constraint parameters, an edge selection that is feasible and yields a favorable objective value under predicted parameters may violate the actual constraints. Some existing approaches incorporate the consequences of infeasibility into their training losses [10, 11], but still rely on optimization solves to obtain the decisions used in training.

A differentiable neural approximation of the parameter-to-decision mapping can support efficient decision feedback and fast decision generation without repeated optimization [12]. When the approximation is differentiable with respect to predicted constraint parameters, losses evaluated under realized parameters can provide feedback on both objective performance and constraint violations [13]. Self-supervised neural optimization methods show that continuous decision representations can be learned from problem objectives and constraints without optimal-solution labels [14, 15]. In DFL, these representations both guide discrete decision construction and provide predictor-training feedback through losses evaluated before construction. The relationship between continuous objectives and discrete solution quality

*Corresponding author
Email address: Xinda.Zheng@monash.edu (X. Zheng)
ORCID(s): 0009-0005-8799-8946 (X. Zheng); 0000-0003-4466-2447 (F.d. Nijs); 0000-0002-4485-5014 (E. Lam)

therefore matters, and depends on the design of the relaxation and rounding procedure [16].

Motivated by these considerations, we propose DFLNOL, a solver-free decision-focused learning framework for NP-hard edge-selection problems with uncertainty in the objective, the constraints, or both. Its self-supervised edge-selection neural optimization layer (NOL) supports predictor training and fast neural deployment without optimal-solution labels or calls to a solver for the original NP-hard problem. We make the following contributions:

- **Solver-free decision-focused learning:** We propose a two-stage pipeline that pretrains the NOL through self-supervision, then trains a predictor through the frozen layer using objective and constraint losses evaluated under observed parameters. At deployment, the predictor, NOL, and a discrete decoder generate decisions without solving the original NP-hard problem.
- **Neural optimization layer for decision-focused learning:** We develop a NOL that combines graph context and current selection features to capture connectivity and competition among edges. Shared iterative refinement, differentiable output transformations, and structural penalties shape a concentrated, structurally consistent continuous representation for predictor training.
- **Decision quality and computational efficiency:** Experiments on asymmetric traveling salesperson (ATSP) and endpoint-free orienteering (EFOP) benchmarks cover objective, constraint, and joint uncertainty. DFLNOL consistently improves decision quality over prediction-focused learning and remains competitive with evaluated solver-based DFL methods, with substantially less training time, including NOL pretraining, and faster neural deployment.

2. Problem formulation

We study edge-selection problems whose objective, constraints, or both depend on problem parameters that are unknown when a decision is made. These parameters are predicted from context, and decision quality is evaluated under their realized values.

2.1. Edge-selection problems

Let $G = (V, E)$ be a directed graph with node set $V = \{1, \dots, n\}$ and edge set $E \subseteq V \times V$, and let $m = |E|$. An edge-selection decision is a binary vector $x \in \{0,1\}^m$, where $x_e = 1$ indicates that edge $e \in E$ is included in the decision.

An instance is specified by G and a parameter vector $u \in \mathbb{R}^d$, formed by concatenating node-level parameters, edge-level parameters, and instance-level parameters, such as rewards, travel costs, and resource limits. With p_v parameters per node, p_e per edge, and p_g at the instance level, the total dimension is $d = np_v + mp_e + p_g$. We leave G implicit in the notation.

Let $c_\gamma(x;u) \le 0$, indexed by a finite set Γ, encode the constraints on the selected edges, such as degree, connectivity, and resource requirements. The feasible set is $\mathcal{X}(u) = \left\{x \in \{0,1\}^m \;\middle|\; c_\gamma(x;u) \le 0 \text{ for all } \gamma \in \Gamma\right\}$, which we assume to be nonempty for every admissible parameter vector u. The objective $F(x;u)$ evaluates a selection under these parameters. Given u, the deterministic optimization problem is to find an optimal solution $x^\star(u) \in \arg\min_{x\in\mathcal{X}(u)} F(x;u)$, where a fixed tie-breaking rule selects $x^\star(u)$ when multiple optimal solutions exist.

2.2. Prediction and optimization

At decision time, the graph and part of the parameter vector are observed. Write $u = (\xi, w)$, where ξ contains the known parameters and w is realized after the decision is committed.

Where w enters the problem determines what is known about feasibility. If it enters only the objective, $\mathcal{X}(u)$ depends on ξ alone and is known at decision time. If it enters the constraints, feasibility also depends on the unknown parameters.

A context vector z contains covariates observed at decision time and used to predict the unknown problem parameters w. A prediction model h_θ, with trainable weights θ, maps the context z to an estimate $\hat{w}_\theta = h_\theta(z)$ of the unknown problem parameters w.

The available historical dataset is $\mathcal{S} = \left\{(u^{(i)}, z^{(i)})\right\}_{i=1}^N$, where each sample records the context and complete realized parameter vector $u^{(i)} = (\xi^{(i)}, w^{(i)})$ of a past instance. The dataset contains no optimal or other reference decisions.

For a new instance with known parameters ξ and context z, prediction followed by optimization gives

$$\hat{u}_\theta = \big(\xi, h_\theta(z)\big), \qquad \hat{x}_\theta = x^\star(\hat{u}_\theta). \tag{1}$$

Predictions are restricted to admissible parameter values, so the estimated problem has a feasible solution. The planned selection $\hat{x}_\theta$ therefore belongs to $\mathcal{X}(\hat{u}_\theta)$, but need not belong to $\mathcal{X}(u)$.

Let $\mathcal{X}_0(\xi)$ denote the binary edge selections satisfying all constraints that are independent of w. Since $\mathcal{X}(\hat{u}_\theta) \subseteq \mathcal{X}_0(\xi)$, the planned selection satisfies the known constraints. When w enters only the objective, $\mathcal{X}_0(\xi) = \mathcal{X}(u)$ and this also ensures feasibility under the realized parameters. Otherwise, the planned selection may require correction.

2.3. Decision-focused learning problem

When uncertain parameters enter the constraints, we consider problems equipped with a prescribed correction rule $\mathrm{corr}(\cdot\,;u) : \mathcal{X}_0(\xi) \longrightarrow \mathcal{X}(u)$. This rule maps a planned edge selection to an executable selection under the realized parameters, respecting the application's restrictions on changes to the plan. It leaves an already feasible selection unchanged: $\mathrm{corr}(x;u) = x$ for every $x \in \mathcal{X}(u)$. The rule is part of the problem specification, and its form depends on the application. When uncertainty affects only the objective, no correction is needed and the rule is the identity map.

For a planned selection $x \in \mathcal{X}_0(\xi)$, we define *regret* by comparing the objective value after correction with the optimal value under the realized parameters:

$$\mathrm{Regret}(x;u) = F\big(\mathrm{corr}(x;u);u\big) - F\big(x^\star(u);u\big). \quad (2)$$

Regret is nonnegative because the corrected selection is feasible, and it is zero exactly when that selection is optimal. For an already feasible x, it reduces to $F(x;u) - F(x^\star(u);u)$. Thus, the consequences of correction are measured through the objective value of the resulting selection.

Decision-focused learning seeks predictions that induce low regret under the joint distribution of (u, z). Its empirical objective on S is $\min_\theta \frac{1}{N}\sum_{i=1}^{N} \mathrm{Regret}\big(\hat{x}_\theta^{(i)};u^{(i)}\big)$, where $\hat{x}_\theta^{(i)}$ is the planned selection defined by (1) for sample i. The learning task is to use contexts and realized parameters, without reference decisions, to obtain predictions whose induced edge selections perform well after any required correction.

3. Related Work

We organize related work around three connected questions: how predictors receive decision feedback, what a learned optimization mapping contributes, and how neural representations of discrete solutions can support predictor training.

3.1. Training feedback in decision-focused learning

DFL methods obtain training feedback through different treatments of the downstream optimization problem. Differentiable optimization layers compute sensitivities through continuous problems [17, 18]. For combinatorial problems, regularized relaxations provide differentiable approximations [3], while MIPaaL tightens a continuous relaxation using cutting planes [19]. DYS-Net improves the efficiency of differentiable optimization by combining Davis–Yin splitting for quadratically regularized relaxations with Jacobian-free backpropagation [20]. Building on this approach, Mandi et al. [21] show that quadratic smoothing can leave large regions with zero decision-loss gradients and combine differentiable optimization with surrogate losses to improve predictor training. Oracle-based methods instead construct training signals from returned solutions, as in black-box differentiation and differentiable perturbations [5, 22]. SPO+ and perturbation-gradient losses similarly use optimization oracles to evaluate surrogate losses and their gradients [9, 23]. These approaches obtain feedback by computing solutions to original or relaxed optimization problems during training. Repeated optimization can remain costly, particularly when the underlying oracle solves an NP-hard problem [4].

Several approaches reduce the cost of obtaining decision feedback. Relaxations and warm starts accelerate optimization calls [4], while contrastive and ranking losses reuse candidate solutions through caching [24, 25]. Geometric losses further reduce online optimization: CaVE uses quadratic-program projections or their approximations to align predicted costs with a cone associated with an optimal reference solution [6], and LAVA computes a solver-free training loss from an optimal vertex and its precomputed adjacent vertices [7]. Beyond reusing solutions or exploiting their geometry, learned loss surrogates approximate decision losses using targets obtained from optimization evaluations. These include locally optimized decision losses and their feature-based extensions [26, 27], as well as LANCER, which alternates between surrogate fitting and predictor updates [28]. These approaches reduce the computation required for predictor feedback, but do not themselves learn a solution mapping for deployment.

The content of the feedback also changes when unknown parameters enter constraints. Hu et al. [29] incorporate solution correction and modification costs into training for packing and covering linear programs. Their subsequent MILP framework formulates correction as a second-stage optimization problem [10]. Odece instead penalizes predictions that induce infeasible solutions or exclude true optimal solutions, using optimization-generated candidates and optimal reference solutions [11]. These studies establish different ways to account for realized feasibility. Our framework evaluates objective and constraint losses on continuous selections produced by a learned mapping, with discrete construction and execution correction outside backpropagation. The mapping therefore provides an interface for both training feedback and deployed solution generation.

3.2. Learned optimization mappings for decision-focused learning

Optimization proxies learn solution mappings from supplied problem parameters. DC3 and primal-dual learning demonstrate self-supervised approximation for continuous constrained optimization without solution labels [30, 31]. ProjectNet extends this approach to predictor training by first learning an optimization module and then training a predictor through the pretrained mapping [12]. Its learned updates use approximate projections onto convex feasible regions, with applications including shortest path and matching. The extended framework also considers two-stage stochastic optimization and differentiation with respect to constraint parameters [13]. An alternative is LtOF, which directly maps context to solutions and is evaluated on continuous optimization problems with objective uncertainty and fixed feasible sets [8]. We retain explicit parameter predictions, allowing the same predictor to support neural or solver-based deployment. Within this interface, we design the NOL to represent interacting edge choices and support discrete solution construction for NP-hard edge-selection problems with objective and constraint uncertainty.

Neural combinatorial optimization offers several representations, with different requirements for integration into predictor training. DIFUSCO learns a diffusion model from optimal or heuristic solution vectors [32], requiring a solution corpus unavailable in our setting. Attention-based

routing models avoid solution labels by using policy gradients over discrete construction trajectories [33]. DIMES introduces a continuous parameterization of solution distributions but retains REINFORCE-based sampling for training and fine-tuning [34]. Thus, continuous parameterization does not necessarily imply direct differentiation of a loss on the represented selection. Our NOL uses this latter mechanism, allowing predictor gradients to pass through its continuous output without differentiating discrete construction.

Self-supervised relaxation methods provide mechanisms for learning continuous representations directly from problem structure. Wang et al. [16] establish conditions under which entry-wise concave relaxations and sequential rounding yield feasible solutions with objective bounds. GON learns node assignments through constraint-aware losses for balanced graph partitioning and maximum independent set [14]. These methods address solution learning and recovery for supplied instances. Their use in contextual edge selection additionally requires a representation of edge connectivity and competition, together with losses that train parameter predictions according to realized objective performance and feasibility.

The edge-selection model of Zheng et al. [15] provides our direct architectural foundation. It combines a cost-attention encoder, self-supervised objective and constraint losses, and lightweight decoding to generate solutions from supplied parameters. We build on this foundation with decision-dependent edge scoring, shared iterative refinement, and problem-specific output transformations. The scorer incorporates relationships derived from the current selection, so each refinement round evaluates edges in the context of surrounding choices. These relationships become part of the continuous computation through which the predictor learns. Our NOL thus extends the earlier solution-generation model into a structured optimization layer for DFL, while retaining discrete decoding for fast deployment.

4. Solver-free decision-focused learning framework

This section presents the framework, explains its two training stages, and describes how the trained models produce decisions at deployment.

4.1. Framework overview

The decision rule in (1) converts predictions into decisions through the solution map $u \mapsto x^\star(u)$. Evaluating this map requires solving the underlying combinatorial optimization problem, as in solver-based DFL methods such as SPO [9]. Its discrete output also obstructs direct differentiation: changes in the predicted parameters can leave the selected decision unchanged, giving zero gradients wherever the map is locally constant. We replace this map with a learned NOL that produces a continuous edge-selection representation.

Given context z, the predictor h_θ estimates the unknown parameters as $\hat{\omega}_\theta = h_\theta(z)$, forming $\hat{u}_\theta = (\xi, \hat{\omega}_\theta)$ together with the known parameters. The NOL, with weights ω, returns $P = \tilde{\pi}_\omega(\hat{u}_\theta) \in [0,1]^m$, where each entry indicates the degree to which an edge is selected. Differentiability with respect to the NOL input allows its output to guide predictor training. A problem-specific decoder converts the continuous representation into a discrete decision at deployment. The NOL architecture is described in Section 5.

Learning proceeds in two stages (Figure 1). Stage 1 trains the NOL on realized parameter vectors using the problem objective and structural requirements. Stage 2 freezes the NOL and trains the predictor through it: predicted parameters generate the continuous selection, while the realized parameters recorded in the training data evaluate its objective performance and any selected constraint violations or structural adjustments. Gradients update the predictor without decoding the selection.

Both stages use the historical data S and the problem specification, without reference decisions or optimal objective values. Model selection compares decoded decision quality on validation data. Deployment combines the trained networks with decoding and, where needed, a solver-free execution rule. The framework therefore avoids calls to the original combinatorial optimization solver during learning and deployment; solver-based methods and reference solutions are used only for benchmarking.

4.2. Training the neural optimization layer

For each realized parameter vector u, the NOL produces $P_\omega = \tilde{\pi}_\omega(u)$. Following the constraint-guided self-supervised approach of Zheng et al. [15], Stage 1 seeks both good objective values and selections consistent with the problem structure. We formulate this as constrained learning, with penalty strengths adjusted according to the deviations observed during training.

Let $\tilde{F}(P;u)$ be a differentiable extension of the objective that agrees with F on binary decisions. For a linear objective, $\tilde{F}(P;u) = \sum_{e\in E} c_e(u) P_e$, with coefficients obtained from the supplied parameters. The objective alone need not favor meaningful selections: selecting no edges, for example, gives zero cost when edge costs are positive, even if the problem requires a nonempty selection.

Training losses may use both the edge-selection output and differentiable intermediate quantities computed in the same NOL evaluation. We leave these intermediate quantities implicit and write the losses as functions of the network parameters and supplied data, rather than of P alone. This introduces no additional data requirement.

Let $\mathcal{K}$ index structural requirements, and let $v_k(\omega;u) \geq 0$ measure the deviation from requirement k in the NOL computation on u, with zero indicating that the condition is satisfied. These differentiable or piecewise-differentiable measures can describe relaxed degree or resource constraints, or proximity to binary selections. They are normalized by problem-dependent scales; for example, budget excess can be expressed relative to the available budget, and local residuals can be averaged over nodes.

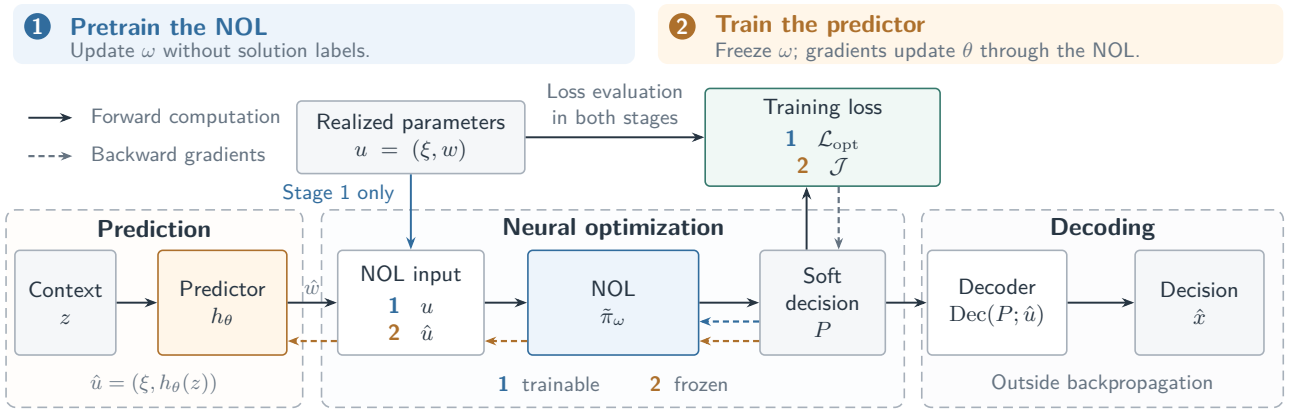


Figure 1: Two-stage training and deployment with a shared neural optimization layer (NOL). Numbered inputs and losses correspond to the two training stages. Intermediate NOL quantities used by the losses are omitted. Deployment follows the horizontal path.

For each requirement, a tolerance $\varepsilon_k \geq 0$ specifies the acceptable average deviation, allowing for numerical and representational limitations. Additional structural preferences without a justified tolerance are included in a differentiable regularizer $\mathcal{R}(\omega; u)$ with fixed coefficients. Unlike the deviation measures, this regularizer need not be nonnegative or vanish on feasible binary decisions.

Positive data-dependent normalization factors may also be applied to the training objective; they are fixed with respect to the trainable parameters and left implicit below. The loss definitions, scales, and any instance-dependent weighting are specified in the case studies.

We reserve a validation subset of $\mathcal{S}$ for model selection. With $\mathbb{E}_{\text{tr}}$ denoting the empirical average over the remaining training instances, the NOL training problem is

$$\begin{aligned} \min_{\omega} \quad & \mathbb{E}_{\text{tr}}\left[\tilde{F}(P_\omega; u) + \mathcal{R}(\omega; u)\right] \\ \text{subject to} \quad & \mathbb{E}_{\text{tr}}\left[v_k(\omega; u)\right] \leq \varepsilon_k, \quad k \in \mathcal{K}. \end{aligned} \tag{3}$$

These conditions constrain average training deviations, not the feasibility of every individual output.

Associating a nonnegative multiplier λ_k with each condition gives the per-instance Lagrangian loss

$$\begin{aligned} \mathcal{L}_{\text{opt}}(\omega; u, \lambda) = \tilde{F}(P_\omega; u) + \mathcal{R}(\omega; u) \\ + \sum_{k \in \mathcal{K}} \lambda_k \left(v_k(\omega; u) - \varepsilon_k\right). \end{aligned} \tag{4}$$

For fixed multipliers, the tolerance terms do not affect the network gradient, but they determine the multiplier updates.

Training uses stochastic primal-dual updates. For a mini-batch $\mathcal{B}_t$, let $\widehat{\mathcal{L}}_t(\omega, \lambda)$ be the batch average of (4) and define $\bar{v}_{k,t} = \frac{1}{|\mathcal{B}_t|} \sum_{u \in \mathcal{B}_t} v_k(\omega_t; u)$. The network follows descent and the multipliers follow projected ascent:

$$\begin{aligned} \omega_{t+1} &= \omega_t - \eta_\omega \nabla_\omega \widehat{\mathcal{L}}_t(\omega_t, \lambda_t), \\ \lambda_{k,t+1} &= \left[\lambda_{k,t} + \eta_{\lambda,k}\left(\bar{v}_{k,t} - \varepsilon_k\right)\right]_+, \quad k \in \mathcal{K}, \end{aligned} \tag{5}$$

where $[a]_+ = \max\{a, 0\}$, $\eta_\omega > 0$ is the network step size, and $\eta_{\lambda,k} > 0$ allows a separate step size for each multiplier. An adaptive gradient optimizer may replace the network descent step. Multipliers are held fixed for that step, and their update history is not differentiated through.

Each multiplier increases when its batch deviation exceeds the tolerance and decreases, subject to nonnegativity, when the deviation falls below it. A zero tolerance therefore permits only increases or no change. Normalizations, tolerances, initial multipliers, and step sizes remain design choices reported in the case studies; these nonconvex updates do not guarantee global optimality or exact feasibility.

After each epoch, outputs on the fixed validation set are decoded under the realized parameters using the procedure in Section 4.4. We retain the checkpoint with the lowest mean objective value of the resulting feasible decisions. This avoids comparing training losses whose multiplier values change over time. During this stage, decoding is used for validation, not for gradient updates.

Let $\omega^\star$ denote the selected weights and write $\tilde{\pi}^\star = \tilde{\pi}_{\omega^\star}$. These weights are frozen for Stage 2, while differentiation with respect to the NOL inputs remains available.

4.3. Training the prediction model

For a training sample (u, z) with $u = (\xi, w)$, the predictor supplies $\hat{u}_\theta = (\xi, h_\theta(z))$ to the frozen NOL, producing $P_\theta = \tilde{\pi}^\star(\hat{u}_\theta)$. Predictions generate the continuous selection; the realized parameters already recorded in the training data determine how it is evaluated.

This distinction matters when uncertainty affects feasibility: a selection satisfying a budget under predicted costs may violate it under realized costs. Stage 1 teaches the NOL to respond to supplied parameters, but cannot eliminate this discrepancy. We therefore define a continuous decision loss $\mathcal{J}(\hat{u}; u)$ that evaluates the frozen NOL computation driven by $\hat{u}$ under the recorded parameters u. Predictor training solves $\min_\theta \ \mathbb{E}_{\text{tr}}\left[\mathcal{J}(\hat{u}_\theta; u)\right]$. The average is over training pairs (u, z).

A general form of the loss, with $P = \tilde{\pi}^\star(\hat{u})$, is

$$\mathcal{J}(\hat{u}; u) = \tilde{F}(P; u) + \sum_{k \in \mathcal{K}_{\text{pred}}} \mu_k v_k^{\text{pred}}(\hat{u}; u) + \mathcal{R}_{\text{pred}}(\hat{u}; u). \tag{6}$$

Here $v_k^{\text{pred}}(\hat{u}; u) \geq 0$ measures a selected constraint violation, such as budget excess under realized resource consumption, and $\mu_k \geq 0$ is its weight. The optional regularizer $\mathcal{R}_{\text{pred}}$ adjusts the continuous evaluation, for example by reducing reward credited to selections that lack structural support. Both terms may use intermediate quantities from the same NOL computation and are differentiable or piecewise differentiable with respect to its input. The recorded parameters u remain fixed: predictions change what the NOL computes, not the cost or resource coefficients used to evaluate it.

The stages serve different purposes: Stage 1 learns useful selections for supplied parameters, whereas Stage 2 learns predictions that induce good selections under realized parameters. Its penalties are therefore task-specific, not copied automatically from Stage 1, and the Stage 1 multipliers are not transferred. The terms, scales, and coefficients are specified in the case studies. Omitting the additional terms recovers objective-only training, $\mathcal{J}(\hat{u}_\theta; u) = \tilde{F}(P_\theta; u)$.

For one training sample, the chain rule gives

$$\nabla_\theta \mathcal{J}(\hat{u}_\theta; u) = \left(\frac{\partial h_\theta(z)}{\partial \theta}\right)^\top \nabla_{\hat{w}_\theta} \mathcal{J}(\hat{u}_\theta; u). \tag{7}$$

The rightmost derivative is with respect to the predicted component of the first argument of $\mathcal{J}$. It includes all paths through the frozen NOL, including intermediate quantities used by the loss, rather than only the final P. Only θ is updated; $\omega^\star$, ξ, and the evaluation parameters u remain fixed. At nondifferentiable points, a subgradient is used. Neither decoding nor execution correction participates in this computation.

This continuous loss is a surrogate, not an exact reformulation of executed regret or the correction procedure in Section 2.3. Its penalties provide a signal about realized violations but do not guarantee feasibility or better decoded decisions. Predictions may also enter regions where the NOL approximation is inaccurate; predictor initialization and output-domain restrictions are specified in the case studies.

Model selection therefore uses executed decision quality. After each epoch, predictions on a fixed validation set pass through the frozen NOL and decoder. We evaluate the resulting decisions under realized parameters, apply the prescribed correction where required, and retain the checkpoint with the lowest mean executed objective value, with parameters denoted by $\theta^\star$. Continuous-output quality and executed quality are assessed separately in the case studies.

4.4. Deployment

At deployment, a new instance provides ξ and z, while w remains unknown. The selected predictor and NOL produce a continuous representation, and a problem-specific decoder $\text{Dec}(P; u)$ constructs a discrete decision under the supplied parameters: $\hat{u} = \big(\xi, h_{\theta^\star}(z)\big)$, $P = \tilde{\pi}^\star(\hat{u})$, and $\hat{x} = \text{Dec}(P; \hat{u})$. This requires one forward pass through each network and one decoding operation, without parameter updates or solving the original optimization problem.

For the settings considered here, the decoder produces a feasible decision for every continuous input when the supplied parameters satisfy its applicability conditions. Predictions must lie in this admissible domain; the conditions, decoding algorithms, and feasibility arguments are given in the case studies. Under these assumptions, $\hat{x} \in \mathcal{X}(\hat{u}) \subseteq \mathcal{X}_0(\xi)$. This guarantee comes from the decoder, not from assuming that P is feasible or ranks edges accurately. Edge rankings still affect objective quality, and feasibility does not imply optimality.

When uncertainty affects only the objective, $\hat{x}$ is also feasible under the realized parameters because the feasible set depends on ξ alone. With uncertain constraints, the rule in Section 2.3 yields $x^{\text{exec}} = \text{corr}(\hat{x}; u) \in \mathcal{X}(u)$, leaving an already feasible decision unchanged. Performance is evaluated by $F(x^{\text{exec}}; u)$. Correction is a separate execution step, not part of planning with unknown parameters, and must itself be solver-free for the complete process to avoid solver calls.

Validation in Stage 2 uses this same prediction, decoding, and execution procedure. The case studies specify the execution rules and evaluate how well the continuous training criterion translates into deployed decision quality.

5. Neural optimization layer for edge selection

This section describes the NOL computation used in Section 4: encoding the input graph, scoring edges using the current selection, and updating that selection through a differentiable transformation.

5.1. Overall architecture and input representation

For a directed graph $G = (V, E)$, the NOL maps supplied parameters to a continuous edge selection. In the architectural formulas below, u denotes the supplied vector: the realized parameters during Stage 1, replaced by $\hat{u}_\theta = (\xi, h_\theta(z))$ during predictor training and deployment. The same computation applies in both cases.

The input comprises node features $\mathbf{a}_i$, edge features $\mathbf{a}_{ij}$, and, where needed, global features $\mathbf{a}_G$. Examples include node rewards, edge travel costs, and an instance-wide budget. Node features may also include statistics of incident edges. Any parameter on which the decision should depend must enter these features or the output transformation; supplying it only to the training loss is insufficient.

Preprocessing may use fixed statistics from the NOL training set or positive, instance-dependent scales computed from the supplied input. Instance-dependent normalization remains on the differentiation path, and quantities with common physical units are scaled consistently. These input transformations are distinct from the objective normalization in Section 4.2; their definitions are specified in the case studies.

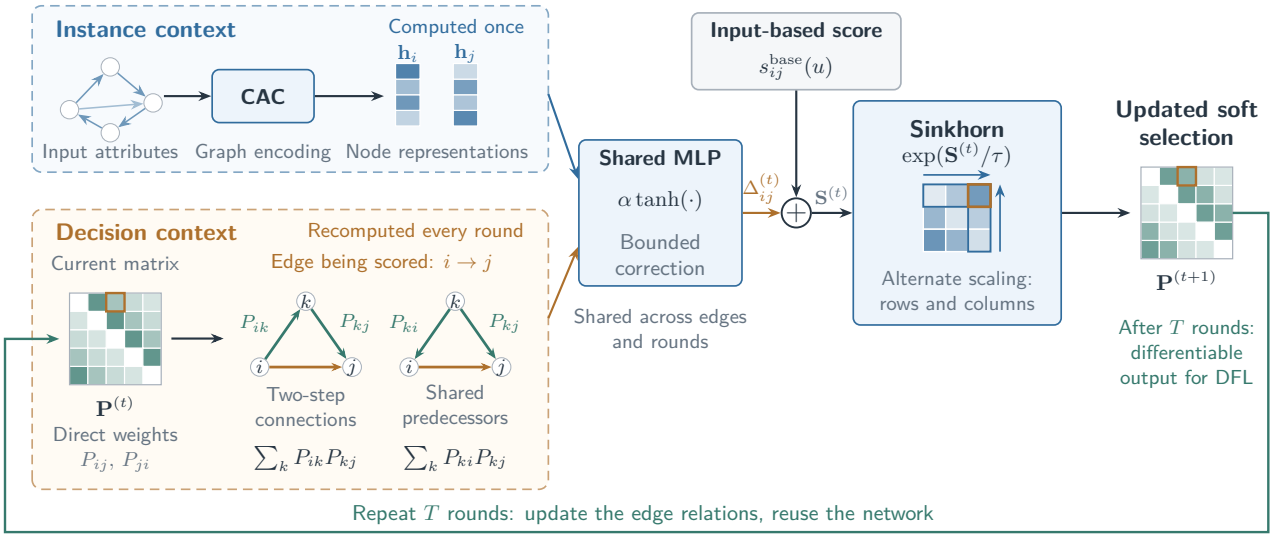


Figure 2: Iterative refinement in the neural optimization layer. Orange highlights the edge (i, j) being scored.

The output is $P = \tilde{\pi}_\omega(u) \in [0,1]^m$. To express interactions between edges, we associate it with a matrix $\mathbf{P} \in [0,1]^{n\times n}$, setting $\mathbf{P}_{ij} = P_e$ for $e = (i,j) \in E$ and zero otherwise. Conversely, $\text{vec}_E(\mathbf{P})$ extracts the edge entries in the ordering of P. These values are continuous selection weights, not necessarily marginal probabilities under a distribution over feasible solutions.

Because the continuous selection enters the training criterion, useful edge rankings alone are insufficient: the weights should also represent compatible choices. For a linear objective with coefficient vector $c(u)$, any soft selection P and feasible binary decision $x \in \mathcal{X}(u)$ satisfy

$$\left|\tilde{F}(P;u) - F(x;u)\right| \le \|c(u)\|_\infty \|P - x\|_1. \tag{8}$$

Proximity to a discrete decision therefore controls the soft-objective error. The bound itself establishes neither feasibility nor optimality; concentration must be considered together with structural consistency and objective quality.

The architecture combines an instance representation computed once with a selection pattern updated over successive rounds (Figure 2). A graph encoder produces node representations, a shared scorer evaluates edges using the current selection, and an output transformation jointly updates their weights.

For a fixed number T of rounds, the computation is

$$\begin{aligned} H &= \text{Enc}_\omega(u), \\ \mathbf{P}^{(0)} &= \mathcal{I}(u), \\ \mathbf{P}^{(t+1)} &= \mathcal{U}_\omega\big(\mathbf{P}^{(t)}, H; u\big). \end{aligned} \tag{9}$$

Here $t = 0, \dots, T-1$, H contains the node representations, $\mathcal{I}$ initializes the soft selection, and $\mathcal{U}_\omega$ combines scoring and the output transformation. The node representations are computed once and reused, without being detached from gradient computation. The selection and its derived features change in each round, while the update parameters are shared. The returned vector is $\tilde{\pi}_\omega(u) = \text{vec}_E\big(\mathbf{P}^{(T)}\big)$.

5.2. Cost-attention convolution

Cost-attention convolution (CAC) incorporates the alternatives around each node into its representation by aggregating neighboring features with cost-dependent weights [15].

The initial node representation is $\mathbf{h}_i^{(0)} = \varphi(W_{\text{in}}\mathbf{a}_i + \mathbf{b}_{\text{in}}) \in \mathbb{R}^{d_h}$, where $\varphi(a) = a/(1+\exp(-a))$ is the elementwise SiLU activation.

Let $\bar{C}_{ij}(u)$ be the normalized cost attribute for edge (i,j). It may represent travel cost or resource consumption and need not equal the objective coefficient $c_e(u)$. For outgoing neighbors $\mathcal{N}^+(i) = \{j : (i,j) \in E\}$, attention weights are

$$A_{ij} = \frac{\exp(-\beta \bar{C}_{ij})}{\sum_{k\in\mathcal{N}^+(i)} \exp(-\beta \bar{C}_{ik})}, \qquad j \in \mathcal{N}^+(i), \tag{10}$$

with zero weights outside the neighborhood. The learned scalar β is initialized positively; when $\beta > 0$, lower-cost neighbors receive larger weights. Nodes without outgoing neighbors receive a zero aggregated message.

The message $\mathbf{m}_i = \sum_{j\in\mathcal{N}^+(i)} A_{ij}\mathbf{h}_j^{(0)}$ is combined with the node's own representation as $\mathbf{h}_i = \varphi(W_{\text{self}}\mathbf{h}_i^{(0)} + W_{\text{neigh}}\mathbf{m}_i + \mathbf{b}_{\text{cac}})$. Stacking these vectors gives $H \in \mathbb{R}^{n\times d_h}$. The encoder weights, including β, belong to ω.

Aggregation follows outgoing edges and imposes no symmetry. CAC describes the supplied instance; the scorer below adds information about the network's current selection.

5.3. Decision-dependent edge scoring

The scorer combines the graph context with the current soft matrix $\mathbf{P}^{(t)}$, allowing an edge to be evaluated in light of the choices currently made around it.

For edge (i, j), $\mathbf{P}^{(t)}_{ij}$ and $\mathbf{P}^{(t)}_{ji}$ give the direct selection weights in both directions, with zero assigned to an absent reverse edge. Two additional relations describe surrounding choices:

$$\begin{aligned} R^{(t)}_{ij} &= \left(\mathbf{P}^{(t)}\mathbf{P}^{(t)}\right)_{ij} = \sum_k \mathbf{P}^{(t)}_{ik}\mathbf{P}^{(t)}_{kj}, \\ M^{(t)}_{ij} &= \left(\mathbf{P}^{(t)\top}\mathbf{P}^{(t)}\right)_{ij} = \sum_k \mathbf{P}^{(t)}_{ki}\mathbf{P}^{(t)}_{kj}. \end{aligned} \tag{11}$$

$R^{(t)}_{ij}$ measures two-step connections $i \to k \to j$. $M^{(t)}_{ij}$ measures weight received by i and j from common predecessors, exposing cases where a node spreads its selection across several destinations. These describe the soft selection pattern, not feasibility violations. The scorer learns whether each relation should increase or decrease an edge's score.

The feature vector supplied to the scorer is

$$\begin{aligned} \mathbf{f}^{(t)}_{ij} = \operatorname{concat}\big(&\mathbf{a}_{ij}, \mathbf{a}_G, \mathbf{h}_i, \mathbf{h}_j, \\ &\mathbf{P}^{(t)}_{ij}, \mathbf{P}^{(t)}_{ji}, \kappa_{\text{rel}} R^{(t)}_{ij}, \kappa_{\text{rel}} M^{(t)}_{ij}\big), \end{aligned} \tag{12}$$

where $\kappa_{\text{rel}} > 0$ is a fixed scale for the relation features. Edge and global features describe the supplied parameters, endpoint representations provide graph context, and the remaining entries describe the current selection. The order of $\mathbf{h}_i$ and $\mathbf{h}_j$ distinguishes source and destination. Unused feature groups are omitted in the problem-specific instantiation.

A shared MLP computes $q^{(t)}_{ij} = \mathbf{w}_2^\top \varphi(W_1 \mathbf{f}^{(t)}_{ij} + \mathbf{b}_1) + b_2$. Its parameters belong to ω and are shared across edges and rounds. The input dimension depends on the supplied features, but the scoring rule retains the same form.

We combine the learned output with a baseline score derived from the instance:

$$\begin{aligned} \Delta^{(t)}_{ij} &= \alpha \tanh\big(q^{(t)}_{ij}\big), \\ S^{(t)}_{ij} &= s^{\text{base}}_{ij}(u) + \Delta^{(t)}_{ij}, \end{aligned} \tag{13}$$

where larger $S^{(t)}_{ij}$ indicates a stronger selection preference. A negative normalized cost, for example, provides a baseline for cost minimization; other problems use a baseline suited to their attributes. The bound $|\Delta^{(t)}_{ij}| < \alpha$ controls how far learning can revise this baseline, so $\alpha > 0$ is chosen together with the feature scales. Every round corrects the same baseline rather than accumulating corrections onto the previous score.

Learned scores guide the NOL computation; they do not replace the recorded objective or constraint coefficients used to evaluate its outputs in either training stage of Section 4.

5.4. Differentiable updates and iterative refinement

Let $\mathbf{S}^{(t)}$ collect the edge scores. The update $\mathbf{P}^{(t+1)} = \mathcal{B}_\tau(\mathbf{S}^{(t)}; u)$ converts them into selection weights in $[0, 1]$, with zero entries outside E. Here $\mathcal{B}_\tau$ is a differentiable, problem-specific transformation and $\tau > 0$ controls its smoothness.

The case studies use Sinkhorn normalization to implement $\mathcal{B}_\tau$, with adaptations to each problem (Figure 2). Alternating row and column normalization couples choices that compete for a common capacity. Balancing may operate on an internal representation, while $\mathbf{P}$ contains only actual edge weights; unit row and column sums are therefore not a universal requirement on $\mathbf{P}$. This distinction allows nodes to remain unselected when the problem permits it. The case studies specify the internal representation and normalization rules.

Lower temperature amplifies score differences in the exponential weights, which can sharpen the selection but also reduce gradients when outputs saturate. It controls concentration and smoothness, not which selection is correct.

Refinement starts from the differentiable initialization $\mathcal{I}(u)$, such as a Softmax transformation of the baseline scores; this initial selection need not satisfy all constraints. Each round recomputes the relations in (11), scores the edges, and applies $\mathcal{B}_\tau$, defining the update $\mathcal{U}_\omega$ in (9). The changed direct edge weights and relation features let the shared scorer reconsider each edge after the preceding update, while the input attributes and node representations are reused.

All rounds and differentiable preprocessing remain on the computation graph. Freezing ω in Stage 2 preserves gradients with respect to the supplied input, including paths through any intermediate quantities used by the loss. This gives the complete input derivative required in (7), without hard selection or decoder calls.

These operations encourage concentrated and structurally consistent selections, but refinement is not a descent algorithm and need not improve the objective at every round. Elementary output constraints also do not ensure full combinatorial feasibility. The case studies assess both the continuous representation and the executed decisions, following the evaluation principles in Section 4.

6. Case studies

This section evaluates the proposed framework on two edge-selection problems. The asymmetric traveling salesperson problem (ATSP) considers uncertainty in the objective, while the endpoint-free orienteering problem (EFOP) considers uncertainty in the constraints or in both the objective and constraints. For each problem, we describe how the framework is instantiated and compare its decision quality and computational cost with baseline methods. We then examine the effects of data characteristics and training-set size.

6.1. Experimental setup

We generate synthetic contextual data using a polynomial construction adopted in previous predict-then-optimize studies [9, 6, 11, 35, 23]. Specifically, we adapt the TSP cost generator of PyEPO [35] to directed edges and use the same cost-generation mechanism for ATSP and EFOP.

For each instance s, we independently sample a contextual vector $z^{(s)} \sim \mathcal{N}(0, I_p)$. Each directed edge (i, j)

has a fixed coefficient vector $b_{ij} \in \mathbb{R}^p$, whose entries are independently generated as the product of a Bernoulli(0.5) variable and a uniform variable on $[-2, 2]$. The edge cost is

$$c_{ij}^{(s)} = \ell_{ij} + \frac{\left(3 + b_{ij}^{\top} z^{(s)}/\sqrt{p}\right)^q}{3^{q-1}} \varepsilon_{ij}^{(s)}, \qquad i \neq j, \quad (14)$$

where ℓ_{ij} is the Euclidean distance between nodes i and j, q is the polynomial degree, and $\varepsilon_{ij}^{(s)} \sim \mathcal{U}(1 - \eta_{\text{noise}}, 1 + \eta_{\text{noise}})$ is independent multiplicative noise. Half of the node coordinates are sampled uniformly from $[-2, 2]^2$, and the remainder from $\mathcal{N}(0, I_2)$. The coordinates and coefficient vectors are sampled once for each dataset and held fixed across its training, validation, and test instances. Reverse edges have independently sampled coefficient vectors and noise, so their costs are generally different. Costs are rounded to four decimal places, and diagonal entries are set to zero. Additional problem-specific parameters are described in the corresponding case study.

The polynomial degree controls the nonlinearity of the contextual mapping, while η_{noise} controls the noise magnitude. For the linear predictors used here, degrees greater than one introduce model misspecification, allowing us to examine whether decision-focused training is beneficial when the predictor cannot exactly represent the underlying relationship. Changing the degree also changes the cost distribution, so it is not interpreted as an isolated measure of optimization difficulty. Both case studies consider $n \in \{50, 100, 200\}$, $p = 10$, $\eta_{\text{noise}} = 0.5$, and $q \in \{4, 6\}$ in the main experiments.

For each setting, training, validation, and test sets are disjoint, with the same split shared by all compared methods. ATSP uses 1,000 training instances per setting, whereas EFOP uses 2,000; both use 64 validation instances and 500 test instances. Each training run starts from a fresh model and optimizer state, without reusing checkpoints from previous experiments. We use model seed 0.

All methods use a linear prediction model with an intercept, $\hat{w}_\theta(z) = Wz + b$, where $z \in \mathbb{R}^p$ is the contextual feature vector and $\hat{w}_\theta(z) \in \mathbb{R}^{d_w}$ contains the predicted unknown parameters [9, 22, 6, 23, 11]. This shared architecture controls for predictive capacity and allows us to evaluate decision quality under model misspecification induced by the nonlinear data-generating process.

The MSE baseline estimates W and b by ordinary least squares. Each decision-focused predictor is initialized from an independently computed least-squares fit on the same training data and subsequently updated using its respective objective. This initialization uses observed parameter values but no optimal decisions. Methods requiring solution labels or optimization oracles retain these components, as specified in each case study.

For DFLNOL, the optimization layer is trained on realized parameters without solution labels and then frozen while the predictor is trained. Both stages use the same training instances. In both problems, the NOL represents each node by a vector of 16 learned features. The edge-scoring MLP has one hidden layer of 32 units and produces a scalar score for each edge. The same scoring network is reused over four refinement rounds, each containing 50 Sinkhorn iterations. The initial edge-selection matrix is computed using a row-wise Softmax with temperature 0.05 for both problems. Each refinement round then applies Sinkhorn normalization with temperature 0.1.

During NOL training, constraint-penalty weights are updated dynamically according to the observed constraint violations, following Section 4.2. This weighting strategy is used for both ATSP and EFOP. Both DFLNOL training stages use Adam with batch size 32, with learning rates of 0.003 for the NOL and $0.001 s_{\text{tr}}$ for the predictor, where s_{tr} is twice the mean off-diagonal realized cost over the corresponding training set. Checkpoints are evaluated on decoded validation decisions under the realized parameters: we minimize mean tour cost for ATSP and maximize mean executed reward for EFOP. We retain the best checkpoint and use a patience of five epochs for early stopping. Model selection requires neither optimal solutions nor optimal objective values. Decoding and any execution correction are used for evaluation, not gradient computation.

We evaluate decision quality using relative regret under the realized parameters. For test instance s, let C_s denote the cost of the ATSP tour produced by a method and R_s the reward collected in EFOP under its execution rule. Let C_s^{ref} and R_s^{ref} denote the corresponding reference values obtained by solving the instance with the realized parameters. Relative regret is computed as $(C_s - C_s^{\text{ref}})/C_s^{\text{ref}}$ for ATSP and $(R_s^{\text{ref}} - R_s)/R_s^{\text{ref}}$ for EFOP, both expressed as percentages. We report the mean and median relative regret to summarize overall decision quality, together with the 95th percentile (P95) and maximum observed regret to assess tail performance.

Training time is measured as wall-clock time for a complete run, including initialization, parameter fitting, validation, and model selection. It also includes required preprocessing, such as generating training solution labels, preparing oracle solutions, or pretraining a NOL. Both stages are included for DFLNOL. Dataset generation, test-reference computation, and hyperparameter searches are excluded. Inference time is the mean time required to produce a discrete decision for one test instance, measured with batch size one after warm-up. It includes prediction and, as applicable, neural optimization and decoding or Gurobi [36] model construction and solution.

Experiments are implemented in Python 3.11 using PyTorch 2.14 and Gurobi 13.0.3 on a high-performance computing cluster. Training and solver jobs are allocated eight CPU cores and 64 GB of RAM on AMD EPYC 7663 nodes, with an NVIDIA A100 GPU with 80 GB of VRAM for GPU-based neural computations.

6.2. Asymmetric traveling salesperson problem

6.2.1. Framework instantiation

The ATSP models applications such as courier delivery, robotic inspection, and cyclic production scheduling, where all required locations or tasks must be visited in a closed

sequence and travel or setup costs depend on the direction of each transition. The problem seeks a minimum-cost directed tour that visits every node exactly once and returns to its starting node. On a complete directed graph without self-loops, it minimizes $\sum_{i\neq j} c_{ij}x_{ij}$ subject to unit incoming and outgoing degrees and the exclusion of subtours. The problem is NP-hard [37]. In our contextual setting, only edge costs are uncertain, so prediction errors affect tour quality but not feasibility.

The NOL maps a cost matrix $D \in \mathbb{R}^{n\times n}$ to a continuous edge-selection matrix $\mathbf{P} \in [0,1]^{n\times n}$, where P_{ij} is the selection weight of directed edge (i,j). Its input is the realized matrix C during NOL pretraining and the predicted matrix $\hat{C}_\theta(z)$ during predictor training and deployment. Contextual features enter the predictor, not the NOL. The continuous output supports differentiation during training and is converted into a feasible tour by a separate decoder.

We normalize the input as $\bar{D}_{ij} = D_{ij}/s(D)$, with $s(D) = \frac{2}{n(n-1)}\sum_{i\neq j}\sqrt{D_{ij}^2+\epsilon_{\text{norm}}^2}$, where $\epsilon_{\text{norm}} > 0$ ensures a positive scale. This differentiable normalization accommodates signed predictions and varying cost magnitudes without changing the ranking of tours under D.

The mean outgoing normalized cost initializes each node's feature, which the CAC encoder maps to a 16-dimensional representation. For each edge (i,j), the scorer combines the two endpoint representations with five values: its normalized cost $\bar{D}_{ij}$, the current forward and reverse selection weights P_{ij} and P_{ji}, and the structural features $(\mathbf{P}^2)_{ij}$ and $(\mathbf{P}^\top\mathbf{P})_{ij}$. Here $(\mathbf{P}^2)_{ij} = \sum_k P_{ik}P_{kj}$ measures two-step connection strength, while $(\mathbf{P}^\top\mathbf{P})_{ij} = \sum_k P_{ki}P_{kj}$ measures selection weight received from common predecessors. These features expose the surrounding selection pattern to the scorer. Together, the endpoint representations and five values form a 37-dimensional input.

A row-wise Softmax of negative normalized costs initializes $\mathbf{P}$. The shared scorer then updates the selection through four refinement rounds, each followed by Sinkhorn normalization. The diagonal is masked throughout, and no symmetry is imposed. Sinkhorn encourages unit row and column sums, but finite iterations can leave degree errors, and even exact balance permits fractional selections and disjoint cycles.

To address these limitations, NOL pretraining combines the soft cost with three structural terms:

$$\mathcal{L}_{\text{NOL}} = \frac{1}{s_{\text{tr}}}\sum_{i\neq j} c_{ij}P_{ij} + \lambda_{\text{deg}}\mathcal{L}_{\text{deg}} + \beta_{\text{con}}\mathcal{L}_{\text{con}} + \lambda_{\text{bin}}\mathcal{L}_{\text{bin}}. \quad (15)$$

The first term favors inexpensive edges and equals the scaled tour cost when $\mathbf{P}$ represents a Hamiltonian tour. The fixed training-set scale s_{tr} controls the loss magnitude and is distinct from the instance-dependent input scale $s(D)$. The degree and binarization multipliers follow Section 4.2, while β_{con} is fixed. Additive tolerance terms independent of the network parameters are omitted.

The degree penalty,

$$\mathcal{L}_{\text{deg}} = \sum_i\left(\sum_{j\neq i}P_{ij}-1\right)^2 + \sum_j\left(\sum_{i\neq j}P_{ij}-1\right)^2, \quad (16)$$

penalizes outgoing and incoming degree errors separately. It complements Sinkhorn normalization by discouraging residual imbalance after a finite number of iterations.

Degree consistency alone does not exclude subtours. We therefore use

$$\mathcal{L}_{\text{con}} = -\log\left|\det\left(\text{Diag}(\mathbf{P1}) - \mathbf{P} + \frac{\mathbf{11}^\top}{n} + \epsilon_{\text{con}}I\right)\right|. \quad (17)$$

The first two terms form a directed graph Laplacian, and the rank-one term removes its zero eigenvalue associated with the constant vector. For a binary cycle cover, additional disconnected cycles introduce further zero eigenvalues. With the small stabilizer $\epsilon_{\text{con}} > 0$, these contribute small factors to the determinant and increase the penalty. This discourages disconnected cycles but is not an exact subtour-elimination constraint.

The binarization penalty is $\mathcal{L}_{\text{bin}} = \sum_{i\neq j}P_{ij}(1-P_{ij})$. Each summand vanishes at 0 and 1 and is positive between them. Combined with the degree conditions, it encourages selection mass to concentrate on individual edges, making the continuous representation closer to a discrete decision. The penalties guide the output structure; feasibility is enforced by decoding.

During predictor training, the NOL is frozen and the loss is $\sum_{i\neq j}c_{ij}(\mathbf{P}_\theta)_{ij}/(ns_{\text{tr}})$, where $\mathbf{P}_\theta$ is induced by $\hat{C}_\theta(z)$. Thus, predictions are evaluated through the continuous output under realized costs, without differentiating through the decoder.

Decoding first constructs a directed cycle cover by solving the assignment problem

$$\max_{\substack{\sigma\in\boldsymbol{\sigma}_n\\ \sigma(i)\neq i\ \forall i}}\sum_i\log\big(\max\{P_{i,\sigma(i)},\delta\}\big), \quad (18)$$

where $\boldsymbol{\sigma}_n$ is the set of node permutations and $\delta > 0$ safeguards the logarithm. Each node then has exactly one successor and one predecessor.

If multiple cycles remain, we merge two by replacing $(a,\sigma(a))$ and $(b,\sigma(b))$ with $(a,\sigma(b))$ and $(b,\sigma(a))$. At each step, we choose nodes in different cycles that minimize the predicted-cost change $\Delta_{ab} = \hat{c}_{a,\sigma(b)} + \hat{c}_{b,\sigma(a)} - \hat{c}_{a,\sigma(a)} - \hat{c}_{b,\sigma(b)}$. Each merge preserves the degree conditions and reduces the cycle count by one, yielding a Hamiltonian tour on the complete graph.

6.2.2. Experimental evaluation

We compare against MSE, SPO+ [9], CaVE [6], PGC [23], and PFYL [22]. MSE is the standard prediction-focused baseline. CaVE uses optimal training solutions to guide cost prediction through cone alignment. PGC uses central differences of optimal objective values, while PFYL learns from optimal training solutions and solutions under

perturbed predicted costs. Methods retain their required labels and oracle calls, whose costs are included in training time.

We evaluate SPO+ with either its standard Gurobi oracle or a neural-oracle variant introduced in this study. SPO+ requires an optimization oracle that maps a supplied cost vector to a discrete tour for computing predictor updates. In our variant, a pretrained NOL followed by decoding replaces this oracle, both when constructing reference tours under the observed costs and when answering shifted-cost queries during training. The NOL remains frozen throughout predictor training. This variant examines whether our learned optimization layer can accelerate an existing DFL method beyond its differentiable use in DFLNOL. Since the neural oracle returns heuristic tours, the exact-oracle guarantees of SPO+ do not directly apply.

Our method is evaluated as DFLNOL, which deploys the NOL with assignment-and-merging decoding, and DFLNOL → Gurobi, which instead solves the predicted ATSP with Gurobi. All baseline methods also use Gurobi at deployment, including SPO+ with the neural training oracle.

Table 1 compares decision quality and computational cost across the six ATSP settings. Solver-based deployment assesses the learned predictions under a common optimization procedure, while DFLNOL additionally evaluates the quality–efficiency trade-off of neural deployment.

Decision-focused training substantially reduces mean regret relative to MSE in all six settings. Without Gurobi deployment, DFLNOL improves both average and upper-tail performance. For example, at 200 nodes and degree 6, mean regret decreases from 178.38% to 21.05%, while P95 decreases from 252.26% to 37.01%. Its advantage over MSE is maintained as the problem size increases: at degree 4, DFLNOL's mean regret rises modestly from 11.62% at 50 nodes to 13.99% at 200 nodes, whereas MSE's increases from 33.19% to 97.84%.

Neural deployment also remains competitive with solver-based DFL. At 200 nodes, DFLNOL achieves lower mean, median, P95, and worst-case regret than SPO+ with a Gurobi oracle for both polynomial degrees, despite requiring no ATSP solver at deployment. PFYL nevertheless achieves lower mean regret in all six settings, so neural deployment does not uniformly outperform solver-based alternatives. With Gurobi deployment, DFLNOL achieves the lowest mean and median regret in every setting and the lowest P95 in five of the six settings. These results indicate that training through the continuous NOL output produces predictions that remain effective under a separate optimization solver, without requiring solver-generated solution labels or ATSP solver calls during either training stage.

DFLNOL has the shortest training time among the decision-focused methods in all six settings, including the cost of NOL pretraining. Even for 200-node instances, training finishes in under two minutes, whereas the competing methods require several minutes to hours. MSE is cheaper to fit but produces substantially higher regret. Neural deployment is also the fastest in every setting, taking less than 15 milliseconds per instance. For 200-node instances, Gurobi-based deployment takes more than one second per instance. The two DFLNOL variants therefore offer a practical trade-off: neural deployment provides fast decisions, while Gurobi deployment achieves lower mean regret at greater inference cost.

The NOL can also accelerate an existing DFL method by replacing its optimization oracle. In SPO+, the pretrained NOL and decoder provide the discrete tours required for predictor updates without repeated Gurobi calls. At 200 nodes and degree 6, this replacement reduces training time from approximately 13 hours to 8 minutes, although mean regret increases from 25.65% to 35.17%. This result suggests that the NOL can serve as a computationally inexpensive approximate oracle within an existing DFL training procedure, with a trade-off between training efficiency and decision quality.

6.3. Endpoint-free orienteering problem

6.3.1. Framework instantiation

The EFOP models applications such as sightseeing itinerary planning, mobile sensing, and selective inspection, where a traveler or agent starts from a given location and selects sites to maximize the collected reward within a travel budget. The problem seeks a maximum-reward directed path starting at a designated node 0, subject to a travel budget B. Each other node may be visited at most once, the endpoint is unrestricted, and no return to the start is required. Let $V^+ = V \setminus \{0\}$ denote the reward-bearing nodes, with $r_0 = 0$. We consider uncertainty in travel costs alone or in both costs and rewards. The budget is known when planning, but uncertain travel costs can make a planned path infeasible under the realized parameters.

The NOL takes a cost matrix, a reward vector, and the budget B as inputs and outputs a continuous edge-selection matrix $\mathbf{P}$. During NOL pretraining, the inputs are the realized parameters (C, r, B). During predictor training and deployment, they are $(\hat{C}_\theta(z), r, B)$ for cost-only prediction and $(\hat{C}_\theta(z), \hat{r}_\theta(z), B)$ for joint prediction. Contextual features z enter the predictor rather than the NOL.

Costs and rewards are normalized by twice their respective instance means, with positive numerical floors. The normalized NOL inputs are denoted by $\bar{c}_{ij}$ and $\bar{r}_i$, using realized or predicted parameters as appropriate to each stage. The cost mean excludes self-loops and edges returning to node 0, while the reward mean excludes node 0. Each node has four initial features: normalized reward, mean outgoing normalized cost, normalized budget $B/(ns_C)$, and a start-node indicator, where s_C is the cost scale of the current input. The CAC encoder maps these features to node representations. For edge (i, j), the scorer combines its two endpoint representations with $\bar{c}_{ij}$, P_{ij}, P_{ji}, $(\mathbf{P}^2)_{ij}$, and $(\mathbf{P}^\top\mathbf{P})_{ij}$. These describe the edge cost, forward and reverse selection weights, two-step connections, and shared predecessors, respectively.

Table 1
ATSP results. Boldface indicates the best performance among decision-focused methods.

Nodes	Degree	Method	Regret (%)				Training time (h:m:s)	Inference time (ms/instance)
			Mean	Median	P95	Worst		
50	4	MSE	33.19	31.33	55.48	123.20	0.14	86.69
		SPO+	9.76	9.45	17.00	37.34	6: 50.91	99.81
		SPO+ (NOL oracle)	12.88	11.67	23.93	48.74	22.52	108.07
		CaVE	10.25	9.80	17.59	38.10	1: 23.70	91.32
		PGC	12.56	11.68	22.44	37.12	12: 52.79	97.36
		PFYL	9.26	8.78	16.09	**29.62**	12: 43.95	92.15
		DFLNOL	11.62	10.89	20.58	39.39	**17.31**	**9.38**
		DFLNOL → Gurobi	**7.97**	**7.65**	**14.33**	39.98	**17.31**	118.51
	6	MSE	59.65	58.22	90.86	148.90	0.08	82.31
		SPO+	14.35	13.10	24.97	49.57	9: 02.13	105.01
		SPO+ (NOL oracle)	18.78	17.40	36.75	62.87	32.74	99.71
		CaVE	12.42	11.50	22.58	65.87	1: 16.97	100.58
		PGC	26.39	25.18	46.04	80.92	24: 23.61	78.36
		PFYL	13.42	12.43	23.69	77.33	15: 42.14	101.27
		DFLNOL	15.46	13.59	27.93	41.51	**26.56**	**9.20**
		DFLNOL → Gurobi	**10.01**	**9.45**	**18.52**	**36.53**	**26.56**	115.54
100	4	MSE	64.81	62.73	111.32	153.93	0.35	356.73
		SPO+	13.19	11.65	24.24	42.89	25: 23.53	372.05
		SPO+ (NOL oracle)	16.36	14.49	28.95	47.63	58.84	365.61
		CaVE	12.50	12.24	18.78	28.40	5: 45.28	375.12
		PGC	18.28	16.41	30.58	51.43	57: 55.36	447.50
		PFYL	11.78	11.13	19.26	27.08	36: 51.84	365.07
		DFLNOL	13.58	12.91	22.81	37.35	**25.45**	**12.04**
		DFLNOL → Gurobi	**9.60**	**9.26**	**15.28**	**24.09**	**25.45**	470.70
	6	MSE	121.79	121.96	176.80	214.40	0.36	443.81
		SPO+	19.17	16.92	34.72	52.75	1:23: 28.58	374.63
		SPO+ (NOL oracle)	26.84	23.83	50.12	79.28	1: 39.59	345.70
		CaVE	32.25	28.76	59.70	104.39	5: 37.89	308.13
		PGC	47.11	44.70	77.92	112.89	1:50: 06.58	339.93
		PFYL	15.04	14.16	24.56	36.46	1:34: 27.38	578.68
		DFLNOL	19.86	17.94	36.35	61.36	**36.01**	**12.10**
		DFLNOL → Gurobi	**12.97**	**11.93**	**24.01**	**35.06**	**36.01**	461.42
200	4	MSE	97.84	92.63	162.17	298.53	1.31	1255.47
		SPO+	16.27	13.98	31.34	114.33	4:17: 06.06	1690.21
		SPO+ (NOL oracle)	20.01	16.60	38.19	119.33	5: 25.72	2798.22
		CaVE	13.92	13.69	18.81	**31.36**	33: 05.28	1858.16
		PGC	26.41	23.10	49.49	121.12	4:45: 28.31	1366.24
		PFYL	12.95	12.21	19.21	53.97	3:28: 07.39	1800.27
		DFLNOL	13.99	13.08	21.71	58.10	**1: 01.27**	**14.21**
		DFLNOL → Gurobi	**10.11**	**9.46**	**16.74**	50.36	**1: 01.27**	2096.04
	6	MSE	178.38	176.43	252.26	356.62	1.36	1223.35
		SPO+	25.65	21.43	52.77	115.47	12:57: 44.75	1721.61
		SPO+ (NOL oracle)	35.17	29.71	70.56	168.13	7: 58.69	2492.82
		CaVE	61.19	55.64	114.48	189.28	37: 12.62	1249.67
		PGC	90.27	88.27	138.40	233.38	8:36: 29.07	1212.22
		PFYL	16.41	15.84	**24.56**	**43.80**	11:03: 53.50	2381.03
		DFLNOL	21.05	19.21	37.01	82.44	**1: 34.41**	**14.49**
		DFLNOL → Gurobi	**14.06**	**12.51**	25.51	63.98	**1: 34.41**	2127.08

A path involves three choices: traveling to another node, leaving a node unvisited, and terminating. We encode these choices in an internal matrix $\mathbf{Q}$. An entry Q_{ij} with $j \in V^+$ and $i \neq j$ represents travel; Q_{ii} for $i \in V^+$ represents skipping node i; and Q_{i0} represents termination at node i. The entry Q_{00} permits an empty path. Termination entries are virtual and incur no return cost. The actual travel matrix has $P_{ij} = Q_{ij}$ for $j \in V^+$ and $i \neq j$, and $P_{ij} = 0$ otherwise. Thus, $y_j = \sum_i P_{ij}$ measures the soft visitation of customer j, rather than requiring every customer to be visited.

The network performs four shared refinement rounds. Actual travel edges receive energies $\bar{c}_{ij} + 0.1 \tanh(f_\omega)$ from the scorer. Skipping node i has energy $\kappa_{\text{skip}} \bar{r}_i$, where $\kappa_{\text{skip}} > 0$ is learned, making it more costly to skip a high-reward node; termination has zero energy. Each round applies Sinkhorn normalization to $\mathbf{Q}$ and extracts $\mathbf{P}$ for the next update. Balancing $\mathbf{Q}$ coordinates travel, skipping, and termination, giving $y_j \approx 1 - Q_{jj}$.

For NOL pretraining, define the soft reward as $R(\mathbf{P}; r) = \sum_{j \in V^+} r_j y_j$ and travel cost as $T(\mathbf{P}; C) = \sum_{i,j} c_{ij} P_{ij}$. The training loss is

$$\mathcal{L}_{\text{NOL}} = -\frac{R(\mathbf{P}; r)}{\sum_{j \in V^+} r_j} + \lambda_{\text{bud}} \mathcal{L}_{\text{bud}} + \lambda_{\text{bal}} \mathcal{L}_{\text{bal}} + \beta_{\text{con}} \mathcal{L}_{\text{con}}, \quad (19)$$

where additive tolerance terms independent of the network parameters are omitted. The budget and balance multipliers follow the dynamic update rule in Section 4.2, while β_{con} is a fixed structural regularization coefficient.

The negative reward term encourages valuable visits. The budget penalty $\mathcal{L}_{\text{bud}} = [T(\mathbf{P}; C)/B - 1]_+$ penalizes expenditure above the budget, where $[a]_+ = \max\{a, 0\}$. The balance penalty is $\mathcal{L}_{\text{bal}} = \frac{1}{n}\left(\|\mathbf{Q}\mathbf{1} - \mathbf{1}\|_2^2 + \|\mathbf{Q}^\top \mathbf{1} - \mathbf{1}\|_2^2\right)$. This discourages residual imbalance after finite Sinkhorn iterations while retaining the option to skip nodes and terminate.

Balanced selections may still contain disconnected cycles whose rewards cannot be collected from the start. Let $\mathbf{P}_+$ be the customer-to-customer submatrix. We use

$$\begin{aligned} \mathcal{L}_{\text{con}} = {} & (n-1)\log(1 + \epsilon_{\text{con}}) \\ & - \log\det\left((1 + \epsilon_{\text{con}})I - \mathbf{P}_+\right). \end{aligned} \quad (20)$$

This term is zero for a binary path because its customer subgraph is acyclic. A disconnected directed cycle introduces an eigenvalue of one, reducing the stabilized determinant and increasing the penalty. The regularizer discourages cycles without requiring skipped nodes to be connected, but does not guarantee path feasibility.

During predictor training, the NOL is frozen, but gradients pass through its continuous output to the predictor. The soft visitation $y_j = \sum_i P_{ij}$ measures incoming selection mass, which does not necessarily indicate that node j is reachable from the start. For example, an isolated cycle can assign full visitation to its nodes without connecting them to node 0. To reduce reward attributed to such selections, we replace y_j in the reward calculation by

$$\tilde{y}_j = \left[\min\{y_j,\ a_0,\ \min_{i \in V^+ \setminus \{j\}} a_{ij}^{\text{ext}}\}\right]_+. \quad (21)$$

Here $a_0 = \sum_{k \in V^+} P_{0k}$ is the mass leaving the start, and $a_{ij}^{\text{ext}} = y_i + y_j - P_{ij} - P_{ji}$ is the total mass entering the node pair $\{i, j\}$ from outside that pair. Thus, a node receives no visitation credit if no mass leaves the start, and an isolated two-node cycle receives no credit because its external incoming mass is zero. The correction only reduces the reward attributed to the output; it does not modify $\mathbf{P}$. These local checks leave valid binary paths unchanged but do not establish connectivity for every continuous selection.

Predicted costs may also underestimate the expenditure required to collect reward. We therefore evaluate the soft travel cost under the realized matrix C and define its budget ratio as $\rho = T(\mathbf{P}_\theta; C)/B$. The predictor minimizes

$$\mathcal{L}_{\text{DFL}} = -\sum_{j \in V^+} r_j \tilde{y}_j + s_R\left(\gamma[\rho - 1]_+^2 + \eta_{\text{cost}}\rho\right), \quad (22)$$

where s_R is the mean total reward in the training set. The first term encourages reward collection after the connectivity correction. The squared penalty is zero within budget and increases with budget excess. The linear term introduces an additional preference for lower expenditure, including below the budget, and therefore trades some reward against travel cost. We use $\gamma = 1$ for cost-only prediction, $\gamma = 3$ for joint prediction, and $\eta_{\text{cost}} = 0.25$. Costs and rewards in this loss are realized values, while the continuous selection is induced by the predicted parameters. Training differentiates through this piecewise-differentiable objective and the frozen NOL, without using the hard decoder.

For neural deployment, a reward-aware greedy decoder starts at node 0. At each step, it selects the unvisited node affordable under the remaining predicted budget with the largest score $s_{ij} = P_{ij}\left(\frac{\hat{r}_j}{\hat{c}_{ij}}\right)$. For cost-only prediction, $\hat{r}_j = r_j$ because rewards are known. The score combines the NOL's edge preference P_{ij} with the reward per unit predicted cost $\hat{r}_j/\hat{c}_{ij}$. Decoding stops when no affordable node remains, without a return edge or local search.

All methods are evaluated using the same execution rule. The planned node order is fixed, and collected reward is computed from the longest prefix whose cumulative realized travel cost does not exceed B. A node earns no reward if it cannot be reached within budget. No reordering or replanning is allowed, so evaluation measures realized reward rather than the reward of a potentially infeasible plan.

6.3.2. Experimental evaluation

We compare against MSE and Odece [11]. MSE predicts the unknown parameters by minimizing prediction error and then solves the predicted problem with Gurobi. Odece is designed for uncertain constraint parameters: its training objective encourages predictions that preserve the feasibility of reference optimal solutions while discouraging decisions that violate the realized constraints. We evaluate Odece in the cost-only setting, where rewards are known. The joint cost-and-reward setting compares MSE with the two DFLNOL deployment variants.

We also evaluate Odece with a neural training oracle. In this variant, a pretrained NOL followed by decoding supplies

the discrete candidate paths used during predictor updates, replacing the corresponding Gurobi calls. The NOL remains frozen, and gradients are not propagated through these discrete paths. Reference optimal solutions are retained, and validation and deployment still use Gurobi. This variant tests whether the NOL can accelerate a constraint-focused DFL method without replacing its training objective.

DFLNOL uses the neural decoder for deployment, while DFLNOL → Gurobi solves the predicted problem with Gurobi. The two variants share the same trained predictor. All methods are evaluated by the reward collected under the realized-cost execution rule in Section 6.3.1. Table 2 reports results for both uncertainty settings across the three problem sizes and two polynomial degrees.

Both DFLNOL variants improve mean, median, P95, and worst-case regret relative to MSE in all twelve settings. The improvement therefore extends beyond average performance to the upper tail. For example, with 200 nodes, degree 4, and uncertain costs, neural deployment reduces mean regret from 52.53% to 14.83% and P95 from 68.81% to 21.82%. The Gurobi-deployed variant also improves on MSE under a common deployment procedure, showing that the benefit is not solely attributable to the neural decoder.

In the cost-only setting, DFLNOL with neural deployment achieves lower mean, median, P95, and worst-case regret than both Odece variants in all six settings. This demonstrates that training through the continuous representation can provide competitive decisions when uncertainty affects feasibility, without requiring optimal training solutions or repeated combinatorial solver calls. Performance nevertheless deteriorates at the higher polynomial degree, indicating that the framework does not eliminate the difficulty of predicting nonlinear contextual relationships.

Neural deployment achieves lower mean regret than Gurobi deployment in eleven of the twelve settings, although Gurobi retains an advantage on some tail metrics. The exception in mean regret is the 200-node, degree-6 joint-prediction setting: Gurobi reduces mean regret slightly from 27.76% to 27.51% and worst-case regret from 66.99% to 64.70%, whereas neural deployment has lower P95 regret (45.14% versus 45.87%). A better solution under predicted parameters need not yield a better executed path under realized costs. These results highlight the importance of evaluating the complete prediction, planning, and execution pipeline.

DFLNOL also has the shortest training time among the evaluated decision-focused methods in every setting, including NOL pretraining. All reported runs finish in under two minutes, whereas Odece requires substantially longer, particularly for larger instances. Neural deployment takes less than 15 milliseconds per instance throughout the table. For the 200-node, degree-6 cost-only setting, it takes approximately 15 milliseconds, compared with approximately 6 seconds for DFLNOL → Gurobi. MSE remains cheaper to train but produces substantially higher regret.

The neural-oracle Odece variant provides a further use of the NOL as an approximate optimization oracle. At 200 nodes and degree 6, replacing the online training oracle reduces total training time from approximately 39 hours to 3.4 hours, while mean regret changes from 37.17% to 36.64%. The effect on decision quality is not uniform across settings, but training time is reduced in every case. This supports the use of the NOL to reduce repeated solver work within an existing DFL procedure, as well as to support direct differentiation.

6.4. Sensitivity analysis

We examine how decision quality depends on the nonlinearity of the contextual mapping, the availability of contextual information, and the amount of training data. Figure 3 compares MSE, DFLNOL, and DFLNOL → Gurobi on 100-node ATSP instances. In panel (a), we vary the polynomial degree over $\{2, 4, 6, 8\}$, using all ten contextual features and 1,000 training instances. In panel (b), we fix the degree at 4 and use 1,000 training instances, but expose only the first $k \in \{2, 4, 6, 8, 10\}$ features to the predictor; costs remain generated from all ten features. This separates missing contextual information from changes in the underlying data-generating process. In panel (c), we fix the degree at 4 and retain all ten features while varying the training-set size over $\{128, 256, 512, 1024, 2048\}$ to assess data efficiency and diminishing returns. Each setting is evaluated on 500 held-out instances using training seed 0, with the same test instances shared by all three methods within each comparison.

In Figure 3(a), the regret of MSE increases substantially with the polynomial degree, whereas DFLNOL exhibits a smaller deterioration over degrees 2–6. With a linear predictor, increasing the degree introduces a mismatch between the predictor and the underlying contextual mapping. The results suggest that optimizing predictions for their downstream decision value can mitigate the consequences of this mismatch more effectively than minimizing prediction error alone. Nevertheless, both DFLNOL variants deteriorate noticeably at degree 8: decision-focused training alleviates, but does not eliminate, the limitations of the predictor. Moreover, changing the degree also changes the cost distribution, so this trend should not be attributed solely to increasing nonlinearity.

Figure 3(b) highlights the role of information availability. When only two contextual features are observed, all three methods exhibit high regret and substantial variability across test instances. As more features become available, regret decreases and the advantage of DFLNOL becomes more pronounced. This suggests that decision-focused training can make effective use of informative context, but cannot fully compensate for missing information. Figure 3(c) provides a complementary perspective on data availability. DFLNOL already outperforms MSE with 128 training instances, and additional data further improves its performance. However, the improvements become progressively smaller as the training set grows, indicating diminishing marginal returns within the evaluated range rather than a definitive saturation point.

Finally, comparing the two DFLNOL deployment variants helps separate the effect of predictor training from that of deployment. DFLNOL → Gurobi outperforms MSE

Table 2
EFOP results. Boldface indicates the best performance among decision-focused methods.

Nodes	Degree	Uncertain components	Method	Regret (%)				Training time (h:m:s)	Inference time (ms/instance)
				Mean	Median	P95	Worst		
50	4	Constraints	MSE	33.19	32.06	51.03	70.66	0.08	251.55
			Odece	21.68	20.36	39.74	58.51	28: 18.94	428.33
			Odece (NOL oracle)	23.14	22.34	38.18	51.91	6: 19.22	468.97
			DFLNOL	**12.61**	**12.14**	**21.85**	**45.70**	**21.77**	**9.74**
			DFLNOL → Gurobi	18.56	17.62	31.14	46.99	**21.77**	247.47
		Constraints and objective	MSE	31.10	30.16	46.35	73.10	0.09	235.91
			DFLNOL	**12.78**	**12.01**	23.55	**47.71**	**23.06**	**9.76**
			DFLNOL → Gurobi	12.87	12.33	**22.10**	49.73	**23.06**	199.49
	6	Constraints	MSE	46.64	45.93	69.60	92.75	0.19	224.86
			Odece	28.32	26.41	53.30	99.37	1:05: 43.24	556.95
			Odece (NOL oracle)	30.52	29.33	52.14	82.87	6: 10.16	479.03
			DFLNOL	**14.52**	**13.49**	**29.49**	**46.36**	**41.68**	**9.59**
			DFLNOL → Gurobi	24.49	22.40	46.38	69.19	**41.68**	236.20
		Constraints and objective	MSE	40.71	40.65	58.43	69.99	0.19	211.14
			DFLNOL	**12.48**	**11.27**	**24.36**	48.06	**1: 15.13**	**9.82**
			DFLNOL → Gurobi	16.22	15.42	26.04	**44.59**	**1: 15.13**	162.15
100	4	Constraints	MSE	44.66	43.77	62.56	81.58	0.32	1047.76
			Odece	24.82	24.08	40.17	55.60	3:29: 21.88	2522.82
			Odece (NOL oracle)	26.26	25.68	40.04	53.82	35: 18.40	2577.08
			DFLNOL	**15.00**	**14.32**	**23.70**	**37.49**	**30.28**	**13.14**
			DFLNOL → Gurobi	20.04	18.59	33.86	50.61	**30.28**	1118.97
		Constraints and objective	MSE	43.14	42.61	58.98	78.10	0.32	1008.55
			DFLNOL	**13.25**	**12.69**	**20.56**	36.36	**47.36**	**12.66**
			DFLNOL → Gurobi	19.99	20.08	25.65	**32.81**	**47.36**	751.51
	6	Constraints	MSE	60.51	60.34	78.60	85.85	0.36	1037.25
			Odece	34.34	32.26	58.90	77.69	8:13: 12.20	4141.82
			Odece (NOL oracle)	32.06	31.26	51.84	75.44	40: 08.30	2401.38
			DFLNOL	**22.00**	**20.75**	**38.97**	**48.72**	**50.48**	**12.47**
			DFLNOL → Gurobi	27.92	26.49	45.79	65.53	**50.48**	1031.86
		Constraints and objective	MSE	56.81	56.69	73.06	88.30	0.31	913.17
			DFLNOL	**23.57**	22.80	**39.64**	**59.90**	**45.07**	**12.64**
			DFLNOL → Gurobi	25.02	**22.35**	47.53	70.62	**45.07**	710.90
200	4	Constraints	MSE	52.53	53.36	68.81	78.63	1.25	5671.09
			Odece	30.38	29.80	47.41	66.96	20:54: 12.98	14651.47
			Odece (NOL oracle)	28.59	28.37	42.14	58.13	2:39: 16.19	12483.19
			DFLNOL	**14.83**	**14.26**	**21.82**	**31.09**	**1: 40.95**	**14.52**
			DFLNOL → Gurobi	21.51	20.85	31.04	43.79	**1: 40.95**	6820.39
		Constraints and objective	MSE	51.52	52.14	66.34	73.55	1.26	5125.83
			DFLNOL	**14.75**	**14.08**	**21.15**	39.27	**1: 14.13**	**14.13**
			DFLNOL → Gurobi	19.38	19.36	25.18	**28.56**	**1: 14.13**	3811.73
	6	Constraints	MSE	68.05	68.45	81.12	88.65	1.28	5433.74
			Odece	37.17	35.31	59.76	80.48	39:12: 31.00	22287.56
			Odece (NOL oracle)	36.64	36.25	54.03	68.86	3:25: 31.70	24603.96
			DFLNOL	**22.65**	**21.95**	**33.83**	**47.75**	**1: 51.38**	**14.54**
			DFLNOL → Gurobi	26.20	24.96	39.85	62.59	**1: 51.38**	6188.14
		Constraints and objective	MSE	64.65	64.97	77.50	85.98	1.25	4455.52
			DFLNOL	27.76	26.45	**45.14**	66.99	**54.83**	**14.18**
			DFLNOL → Gurobi	**27.51**	**26.42**	45.87	**64.70**	**54.83**	4195.23

despite both using Gurobi at deployment, indicating that the benefit is not solely attributable to the neural decoder. Meanwhile, neural deployment retains much of this improvement without invoking Gurobi. Together, these observations support the usefulness of the NOL both as a differentiable training component and as part of the deployment pipeline.

7. Conclusion and future development

This paper presented DFLNOL, a solver-free decision-focused learning framework for edge-selection problems. A self-supervised neural optimization layer provides a continuous decision representation through which predictors learn from objectives and constraint penalties evaluated under

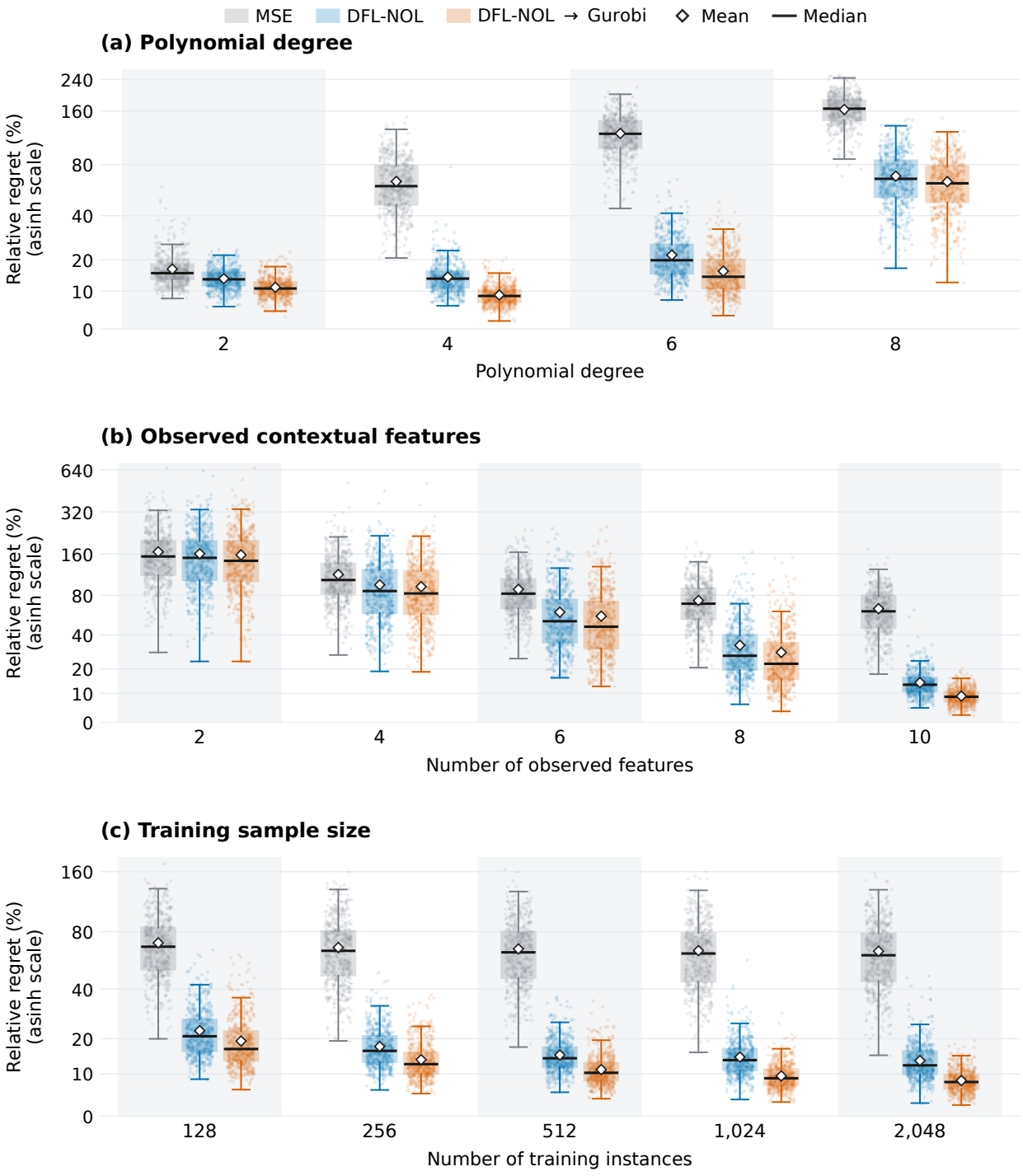


Figure 3: Sensitivity analysis on ATSP with 100 nodes. Dots show individual test instances within the displayed ranges. Boxes show interquartile ranges, horizontal lines indicate medians, white diamonds indicate means, and whiskers mark the 5th and 95th percentiles.

realized parameters. The framework accommodates uncertainty in objectives, constraints, or both, without requiring optimal-solution labels or calls to the original NP-hard solver during training and neural deployment. Experiments on ATSP and EFOP demonstrate lower regret than MSE and a favorable trade-off between decision quality and computational cost compared with the evaluated DFL baselines. The

layer also reduces training cost when used as an approximate oracle within existing DFL methods.

Future work will focus on improving the agreement between continuous training losses and executed decision quality, particularly when predictor updates shift inputs beyond the optimization layer's training distribution. Extending the framework to additional problem classes and evaluating it on real-world data and larger instances will help establish its broader applicability and robustness.

References

[1] U. Sadana, A. Chenreddy, E. Delage, A. Forel, E. Frejinger, T. Vidal, A survey of contextual optimization methods for decision-making under uncertainty, European Journal of Operational Research 320 (2025) 271–289.

[2] J. Mandi, J. Kotary, S. Berden, M. Mulamba, V. Bucarey, T. Guns, F. Fioretto, Decision-focused learning: Foundations, state of the art, benchmark and future opportunities, Journal of Artificial Intelligence Research 80 (2024) 1623–1701.

[3] B. Wilder, B. Dilkina, M. Tambe, Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 33, 2019, pp. 1658–1665. doi:10.1609/aaai.v33i01.33011658.

[4] J. Mandi, E. Demirović, P. J. Stuckey, T. Guns, Smart predict-and-optimize for hard combinatorial optimization problems, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, 2020, pp. 1603–1610. doi:10.1609/aaai.v34i02.5521.

[5] M. Vlastelica, A. Paulus, V. Musil, G. Martius, M. Rolínek, Differentiation of blackbox combinatorial solvers, in: International Conference on Learning Representations, 2020. doi:10.48550/arXiv.1912.02175.

[6] B. Tang, E. B. Khalil, CaVE: A cone-aligned approach for fast predict-then-optimize with binary linear programs, in: Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR 2024), volume 14743 of *Lecture Notes in Computer Science*, 2024, pp. 193–210. doi:10.1007/978-3-031-60599-4_12.

[7] S. Berden, A. İ. Mahmutoğulları, D. Tsouros, T. Guns, Solver-free decision-focused learning for linear optimization problems, in: Advances in Neural Information Processing Systems, volume 38, 2025, pp. 158127–158145. doi:10.52202/085713-5284.

[8] J. Kotary, V. Di Vito, J. Christopher, P. Van Hentenryck, F. Fioretto, Learning joint models of prediction and optimization, in: ECAI 2024 – 27th European Conference on Artificial Intelligence, volume 392 of *Frontiers in Artificial Intelligence and Applications*, IOS Press, 2024, pp. 2476–2483. doi:10.3233/FAIA240775.

[9] A. N. Elmachtoub, P. Grigas, Smart "predict, then optimize", Management Science 68 (2022) 9–26.

[10] X. Hu, J. C. H. Lee, J. H. M. Lee, Two-stage predict+Optimize for MILPs with unknown parameters in constraints, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 14247–14272. URL: https://papers.neurips.cc/paper_files/paper/2023/file/2e14be0332c04c76742710e417cedb2a-Paper-Conference.pdf. doi:10.52202/075280-0627.

[11] J. Mandi, M. Defresne, S. Berden, T. Guns, Feasibility-aware decision-focused learning for predicting parameters in the constraints, in: Advances in Neural Information Processing Systems, volume 38, 2025, pp. 163285–163309. doi:10.52202/085713-5451.

[12] R. Cristian, P. Harsha, G. Perakis, B. L. Quanz, I. Spantidakis, End-to-end learning for optimization via constraint-enforcing approximators, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 37, 2023, pp. 7253–7260. doi:10.1609/aaai.v37i6.25884.

[13] R. Cristian, P. Harsha, G. Perakis, B. Quanz, Efficient end-to-end learning for decision-making: A meta-optimization approach, 2025. doi:10.48550/arXiv.2505.11360. arXiv:2505.11360.

[14] C. Liu, J. Wang, Y. Cao, M. Liu, W. Shen, GON: End-to-end optimization framework for constraint graph optimization problems, Knowledge-Based Systems 254 (2022) 109697.

[15] X. Zheng, F. de Nijs, E. Lam, Constraint-aware self-supervised learning for edge selection, in: 32nd International Conference on Principles and Practice of Constraint Programming (CP 2026), volume 379 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 2026, pp. 61:1–61:18. doi:10.4230/LIPIcs.CP.2026.61.

[16] H. P. Wang, N. Wu, H. Yang, C. Hao, P. Li, Unsupervised learning for combinatorial optimization with principled objective relaxation, in: Advances in Neural Information Processing Systems, volume 35, 2022, pp. 31444–31458. doi:10.52202/068431-2280.

[17] B. Amos, J. Z. Kolter, OptNet: Differentiable optimization as a layer in neural networks, in: Proceedings of the 34th International Conference on Machine Learning, volume 70 of *Proceedings of Machine Learning Research*, 2017, pp. 136–145. doi:10.48550/arXiv.1703.00443.

[18] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, J. Z. Kolter, Differentiable convex optimization layers, in: Advances in Neural Information Processing Systems, volume 32, 2019. doi:10.48550/arXiv.1910.12430.

[19] A. Ferber, B. Wilder, B. Dilkina, M. Tambe, MIPaaL: Mixed integer program as a layer, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, 2020, pp. 1504–1511. doi:10.1609/aaai.v34i02.5509.

[20] D. McKenzie, S. Wu Fung, H. Heaton, Differentiating through integer linear programs with quadratic regularization and Davis–Yin splitting, Transactions on Machine Learning Research (2024).

[21] J. Mandi, A. İ. Mahmutoğulları, S. Berden, T. Guns, Minimizing surrogate losses for decision-focused learning using differentiable optimization, in: ECAI 2025, 2025. doi:10.3233/FAIA251273.

[22] Q. Berthet, M. Blondel, O. Teboul, M. Cuturi, J.-P. Vert, F. Bach, Learning with differentiable perturbed optimizers, in: Advances in Neural Information Processing Systems, volume 33, 2020, pp. 9508–9519. doi:10.48550/arXiv.2002.08676.

[23] V. Gupta, M. Huang, Decision-focused learning with directional gradients, in: Advances in Neural Information Processing Systems, volume 37, 2024, pp. 79194–79220. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/907a9fb75a408f6c3a2ae1bf84c39e44-Abstract-Conference.html. doi:10.52202/079017-2514.

[24] M. Mulamba, J. Mandi, M. Diligenti, M. Lombardi, V. Bucarey, T. Guns, Contrastive losses and solution caching for predict-and-optimize, in: Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, 2021, pp. 2833–2840. doi:10.24963/ijcai.2021/390.

[25] J. Mandi, V. Bucarey, M. M. K. Tchomba, T. Guns, Decision-focused learning: Through the lens of learning to rank, in: Proceedings of the 39th International Conference on Machine Learning, volume 162 of *Proceedings of Machine Learning Research*, 2022, pp. 14935–14947. URL: https://proceedings.mlr.press/v162/mandi22a.html. doi:10.48550/arXiv.2112.03609.

[26] S. Shah, K. Wang, B. Wilder, A. Perrault, M. Tambe, Decision-focused learning without decision-making: Learning locally optimized decision losses, in: Advances in Neural Information Processing Systems, volume 35, 2022, pp. 1320–1332. doi:10.52202/068431-0097.

[27] S. Shah, B. Wilder, A. Perrault, M. Tambe, Leaving the nest: Going beyond local loss functions for predict-then-optimize, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, 2024, pp. 14902–14909. doi:10.1609/aaai.v38i13.29410.

[28] A. Zharmagambetov, B. Amos, A. Ferber, T. Huang, B. Dilkina, Y. Tian, Landscape surrogate: Learning decision losses for mathematical optimization under partial information, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 27332–27350. doi:10.52202/075280-1191.

[29] X. Hu, J. C. H. Lee, J. H. M. Lee, Predict+optimize for packing and covering LPs with unknown parameters in constraints, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 37, 2023, pp. 3987–3995. doi:10.1609/aaai.v37i4.25513.

[30] P. L. Donti, D. Rolnick, J. Z. Kolter, DC3: A learning method for optimization with hard constraints, in: International Conference

on Learning Representations (ICLR), 2021. doi:10.48550/arXiv.2104.12225. arXiv:2104.12225.

[31] S. Park, P. Van Hentenryck, Self-supervised primal-dual learning for constrained optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence, volume 37, 2023, pp. 4052–4060. doi:10.1609/aaai.v37i4.25520.

[32] Z. Sun, Y. Yang, DIFUSCO: Graph-based diffusion solvers for combinatorial optimization, in: Advances in Neural Information Processing Systems, volume 36, 2023, pp. 3706–3731. doi:10.52202/075280-0164.

[33] W. Kool, H. van Hoof, M. Welling, Attention, learn to solve routing problems!, in: International Conference on Learning Representations, 2019. doi:10.48550/arXiv.1803.08475.

[34] R. Qiu, Z. Sun, Y. Yang, DIMES: A differentiable meta solver for combinatorial optimization problems, in: Advances in Neural Information Processing Systems, volume 35, 2022, pp. 25531–25546. doi:10.52202/068431-1851.

[35] B. Tang, E. B. Khalil, PyEPO: A PyTorch-based end-to-end predict-then-optimize library for linear and integer programming, Mathematical Programming Computation 16 (2024) 297–335.

[36] Gurobi Optimization, LLC, Gurobi optimizer reference manual, 2024. URL: https://www.gurobi.com.

[37] R. Roberti, P. Toth, Models and algorithms for the Asymmetric Traveling Salesman Problem: an experimental comparison, EURO Journal on Transportation and Logistics 1 (2012) 113–133.